

Experiment Report

Course: Computer Programming

Experiment 3: Functions

Class:

Name:

Student ID:

Instructor:

Date: 2026.6.18

Experiment 3: Functions

I. Objectives and Content

This experiment aims to practice C++ functions, which is chapter 6 of the textbook. The detailed goals of this experiment are as follows.

- **Modularity:** Understand the benefits of dividing a program into smaller, manageable tasks using functions.
- **Function Elements:** Master the three essential parts: Function Declaration (Prototype), Definition, and Call.
- **Data Exchange:** Understand the mechanism of passing parameters and returning values.
- **Library vs. User-defined:** Utilize C++ standard library functions (math, cmath) and create custom user-defined functions.
- **Scope & Lifetime:** Distinguish between local and global variables and understand their visibility in memory.

Complete the following 8 programming tasks. Ensure all code includes proper documentation and follows the specific technical requirements for each problem.

II. Preparation

This preparation focuses on reviewing the basic knowledge and programming skills required for Experiment 3. Before completing the programming tasks, the relevant concepts of C++ functions were previewed, including function declaration, function definition, function call, parameter passing, return values, and variable scope. The preparation also included reviewing the use of standard library functions from `<cmath>` and formatted output with `<iomanip>`. In addition, the Visual Studio Code environment and C++ compiler were checked to ensure that each program could be compiled and executed correctly. Through this preparation, the logical structure, input and output requirements, and testing methods for each task were clarified.

III. Project Results

Task 1: Greeting Function (Void, No Parameters)

1. **Description:** Describe the program logic in words or with a flow diagram. Do not paste the source code here.

- 1) Write the void function displayHeader() that prints my name and lab info.

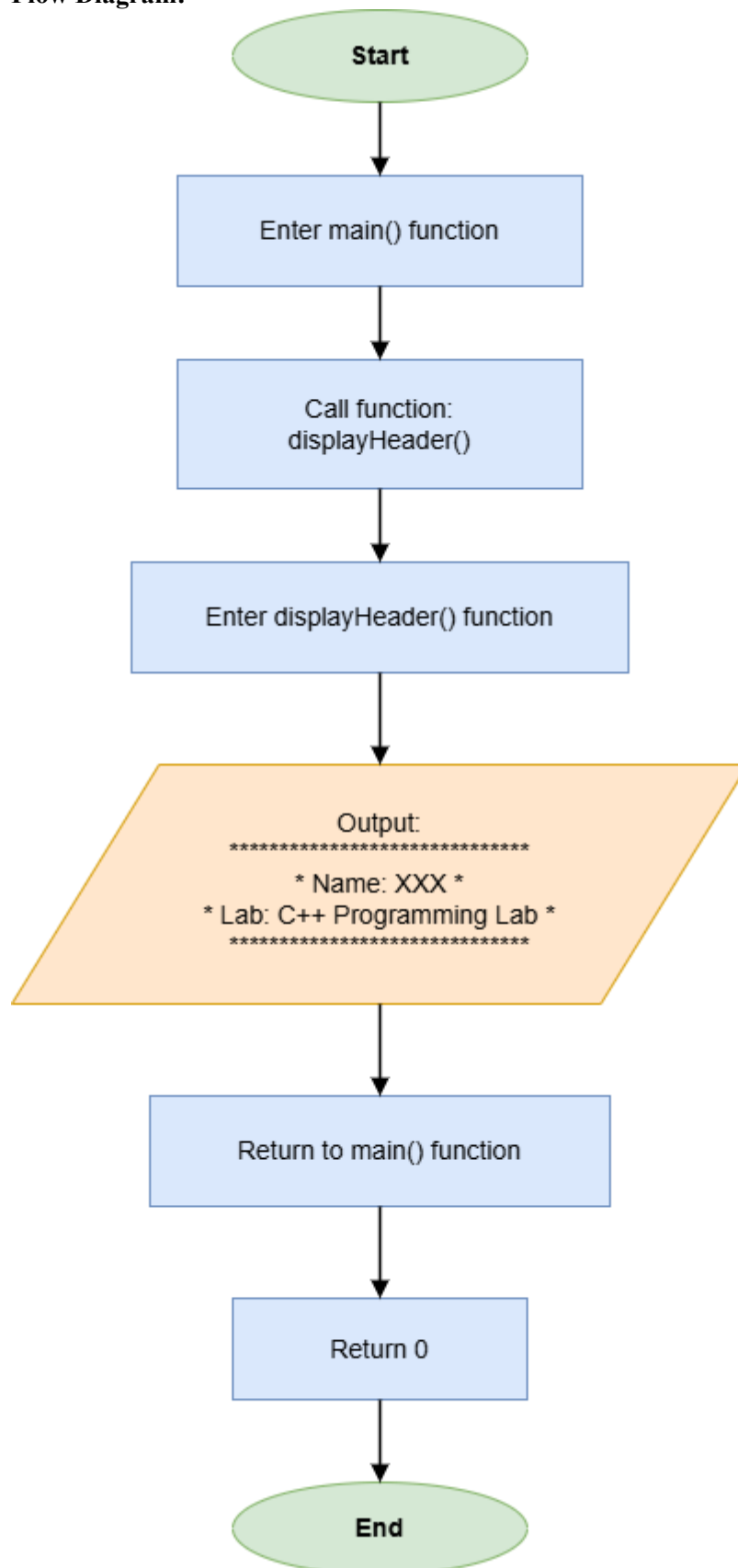
```
void displayHeader()
{
    cout << "*****" << endl;
    cout << "*Name: XXX          *" << endl;
    cout << "*Lab: C++ Programming Lab   *" << endl;
    cout << "*****" << endl;
}
```

- 2) Call it in the main function.

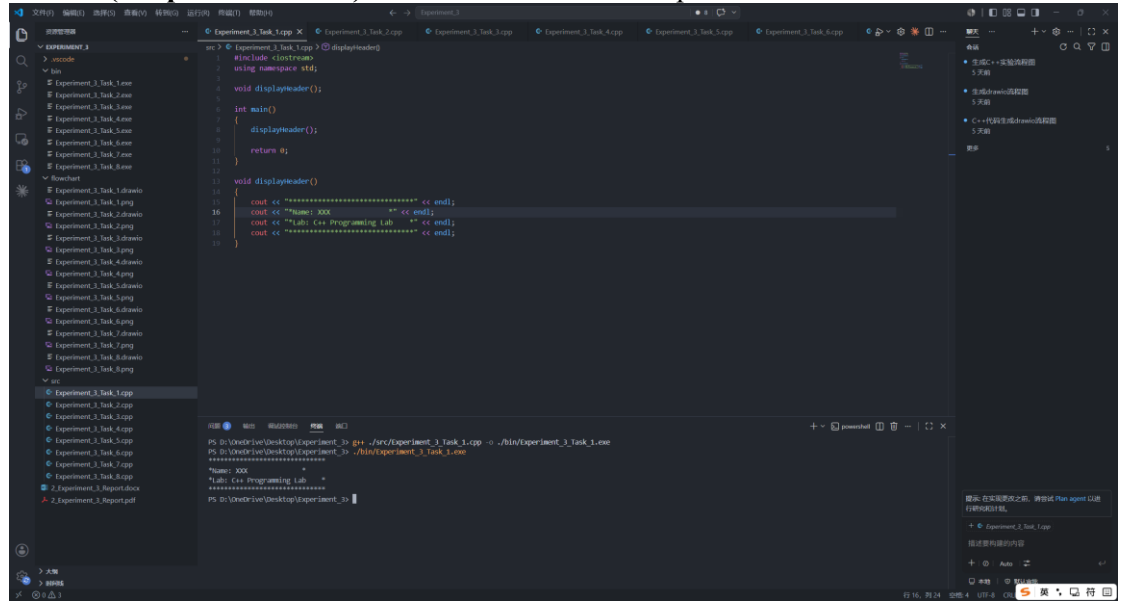
```
int main()
{
    displayHeader();

    return 0;
}
```

2. Flow Diagram:



3. Results (Output Screenshot): The screenshot of the output of the code.



The screenshot displays a C++ development environment with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'EXPERIMENT_3' with various source files. The code editor shows the following C++ code:

```
#include <iostream>
using namespace std;

void displayHeader();

int main()
{
    displayHeader();
    return 0;
}

void displayHeader()
{
    cout << "*****" << endl;
    cout << "Name: XXX" << endl;
    cout << "Lab: C++ Programming Lab" << endl;
    cout << "*****" << endl;
}
```

The terminal at the bottom shows the command prompt output:

```
PS D:\OneDrive\Desktop\Experiment_3> g++ ./src/Experiment_3_Task_1.cpp -o ./bin/Experiment_3_Task_1.exe
PS D:\OneDrive\Desktop\Experiment_3> ./bin/Experiment_3_Task_1.exe
Name: XXX
Lab: C++ Programming Lab
*****
```

Task 2: Square and Cube (Data-Returning, One Parameter)

1. Description:

- 1) Create a function long calculatePower(int base, int exp) that returns $base^{exp}$.

```
long calculatePower(int base, int exp)
{
    long result = 1;
    for (int i = 0; i < exp; i++)
    {
        result *= base;
    }
    return result;
}
```

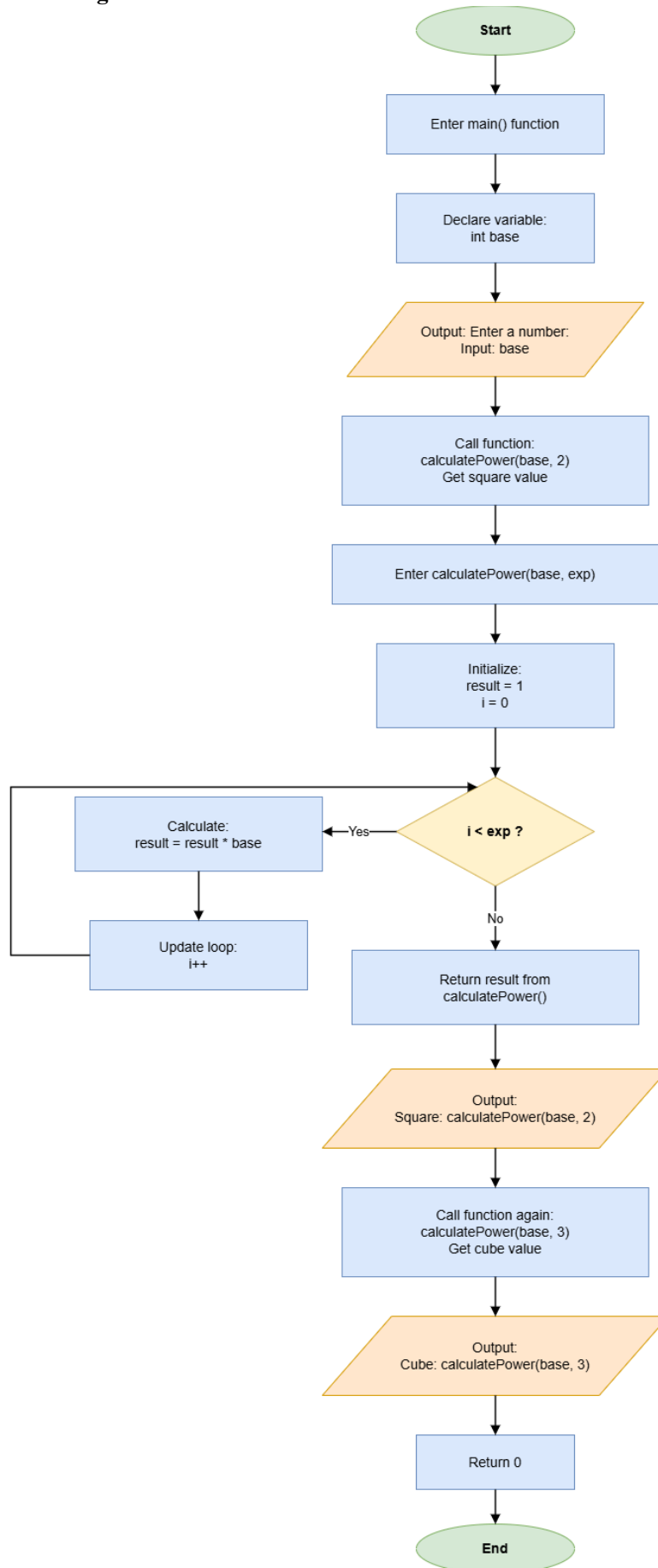
- 2) Input the number base.

```
int base;
cout << "Enter a number: ";
cin >> base;
```

- 3) Print the square and the cube.

```
cout << "Square: " << calculatePower(base, 2) << endl;
cout << "Cube: " << calculatePower(base, 3) << endl;
```

2. Flow Diagram:



3. Results (Output Screenshot):

The screenshot displays the Visual Studio Code interface with the following components:

- File Explorer (Left):** Shows a project structure with folders like `bin` and `src`, and files including `Experiment_3_Task_1.exe` through `Experiment_3_Task_8.exe`, and `2_Experiment_Report.docx`.
- Source Code (Center):** The file `Experiment_3_Task_2.cpp` is open, showing a C++ program that calculates the power of a number. The code includes headers, a `calculatePower` function, and a `main` function that prompts the user for a base and an exponent, then prints the square and cube of the base.
- Output Console (Bottom):** Displays the execution results:

```
PS D:\OneDrive\Desktop\Experiment_3> .\bin\Experiment_3_Task_2.exe
Enter a number: 3
Square: 9
Cube: 27
PS D:\OneDrive\Desktop\Experiment_3>
```
- Search (Right):** Shows search results for the code, including a snippet for `calculatePower`.

Task 3: Maximum of Three (Comparison Logic)

1. Description:

- 1) Write a function `int findMax(int n1, int n2, int n3)` that returns the largest of the three integers.

```
int findMax(int n1, int n2, int n3)
{
    if (n1 >= n2 && n1 >= n3)
    {
        return n1;
    }
    else if (n2 >= n1 && n2 >= n3)
    {
        return n2;
    }
    else
    {
        return n3;
    }
}
```

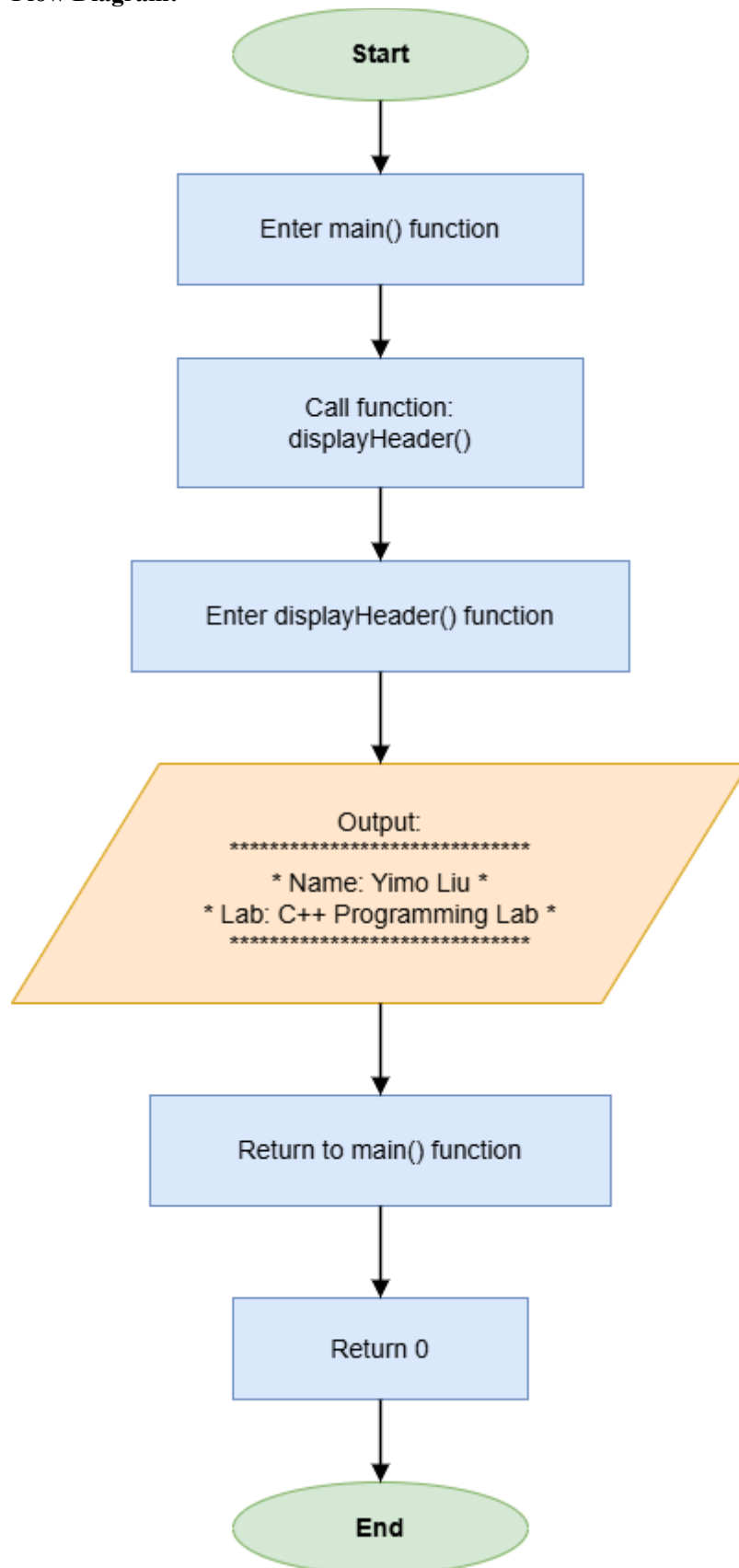
- 2) Input the three integers.

```
int n1, n2, n3;
cout << "Enter three numbers: ";
cin >> n1 >> n2 >> n3;
```

- 3) Call `findMax()` from main and display the returned maximum value.

```
cout << "Maximum value: " << findMax(n1, n2, n3) << endl;
```

2. Flow Diagram:



3. Results (Output Screenshot):

The screenshot displays the Visual Studio Code interface with a C++ project named 'EXPERIMENT_3'. The file explorer on the left shows a directory structure with various task files and a report document. The main editor window displays the source code for 'Experiment_3_Task_3.cpp', which includes a `findMax` function and a `main` function. The `main` function prompts the user to enter three numbers and prints the maximum value found. The output window at the bottom shows the execution results, indicating that the program ran successfully and printed the maximum value of 25.

```
src > Experiment_3_Task_3.cpp > findMax(int, int, int)
1  #include <iostream>
2  using namespace std;
3
4  int findMax(int n1, int n2, int n3);
5
6  int main()
7  {
8      int n1, n2, n3;
9      cout << "Enter three numbers: ";
10     cin >> n1 >> n2 >> n3;
11
12     cout << "Maximum value: " << findMax(n1, n2, n3) << endl;
13
14     return 0;
15 }
16
17 int findMax(int n1, int n2, int n3)
18 {
19     if (n1 >= n2 && n1 >= n3)
20     {
21         return n1;
22     }
23     else if (n2 >= n1 && n2 >= n3)
24     {
25         return n2;
26     }
27     else
28     {
29         return n3;
30     }
31 }
```

Output:

```
PS D:\OneDrive\Desktop\Experiment_3> ./bin/Experiment_3_Task_3.exe
Enter three numbers: 18 25 7
Maximum value: 25
PS D:\OneDrive\Desktop\Experiment_3>
```

Task 4: Circle Area Calculator (Constants & Math)

1. Description:

- 1) Write a function `double getCircleArea(double radius)` that returns the area of a circle using $A = \pi r^2$.

```
double getCircleArea(double radius)
{
    const double PI = 3.14159;
    return PI * radius * radius;
}
```

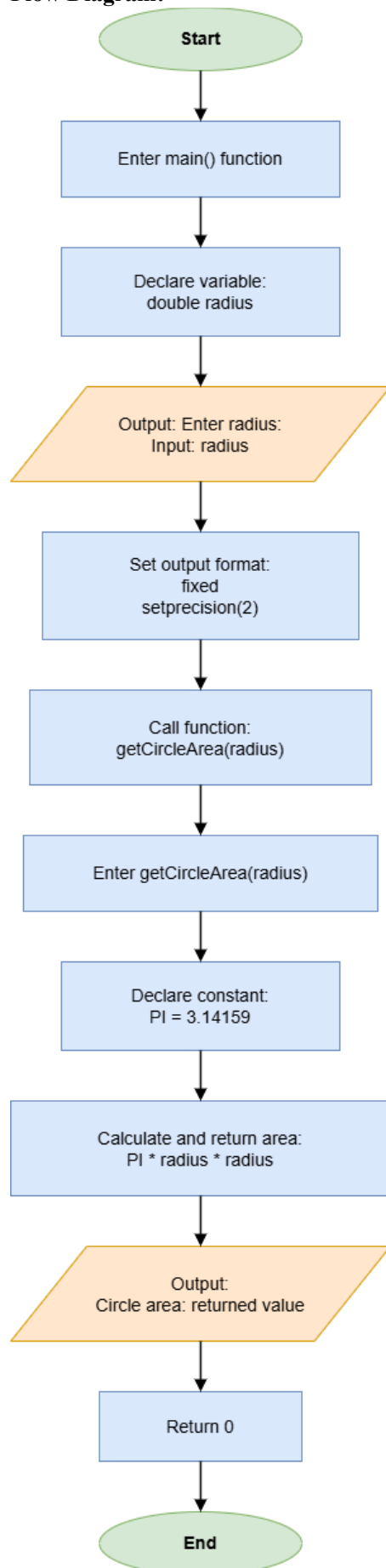
- 2) Input the radius.

```
double radius;
cout << "Enter radius: ";
cin >> radius;
```

- 3) Call `getCircleArea()` from `main()` and print the area of the circle with `setprecision(2)`.

```
cout << fixed << setprecision(2);
cout << "Circle area: " << getCircleArea(radius) << endl;
```

2. Flow Diagram:



3. Results (Output Screenshot):

The screenshot displays the Visual Studio Code interface with a C++ project named 'Experiment_3'. The file explorer on the left shows a directory structure with 'src' containing several task files. The main editor window shows the code for 'Experiment_3_Task_4.cpp'. The code defines a function 'getCircleArea' and a 'main' function that prompts the user for a radius and prints the calculated area. The output window at the bottom shows the execution results for 'Experiment_3_Task_4.exe', indicating that the radius entered was 5 and the resulting circle area was 78.54.

```
src > Experiment_3_Task_4.cpp > getCircleArea(double)
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  double getCircleArea(double radius);
6
7  int main()
8  {
9      double radius;
10     cout << "Enter radius: ";
11     cin >> radius;
12
13     cout << fixed << setprecision(2);
14     cout << "Circle area: " << getCircleArea(radius) << endl;
15
16     return 0;
17 }
18
19 double getCircleArea(double radius)
20 {
21     const double PI = 3.14159;
22     return PI * radius * radius;
23 }
```

Output:

```
PS D:\OneDrive\Desktop\Experiment_3> ./bin/Experiment_3_Task_4.exe
Enter radius: 5
Circle area: 78.54
PS D:\OneDrive\Desktop\Experiment_3>
```

Task 5: Temperature Converter (Multiple Functions)

1. Description:

- 1) Create two functions: `celsiusToFahrenheit()` and `fahrenheitToCelsius()`.

```
double celsiusToFahrenheit(double celsius)
{
    return (celsius * 9.0 / 5.0) + 32;
}

double fahrenheitToCelsius(double fahrenheit)
{
    return (fahrenheit - 32) * 5.0 / 9.0;
}
```

- 2) Print the menu and allow the user to choose and input the temperature.

```
cout << "Select conversion:" << endl;
cout << "1. Celsius to Fahrenheit" << endl;
cout << "2. Fahrenheit to Celsius" << endl;

int choice, temperature;
cout << "Enter choice: ";
cin >> choice;
cout << "Enter temperature: ";
cin >> temperature;
```

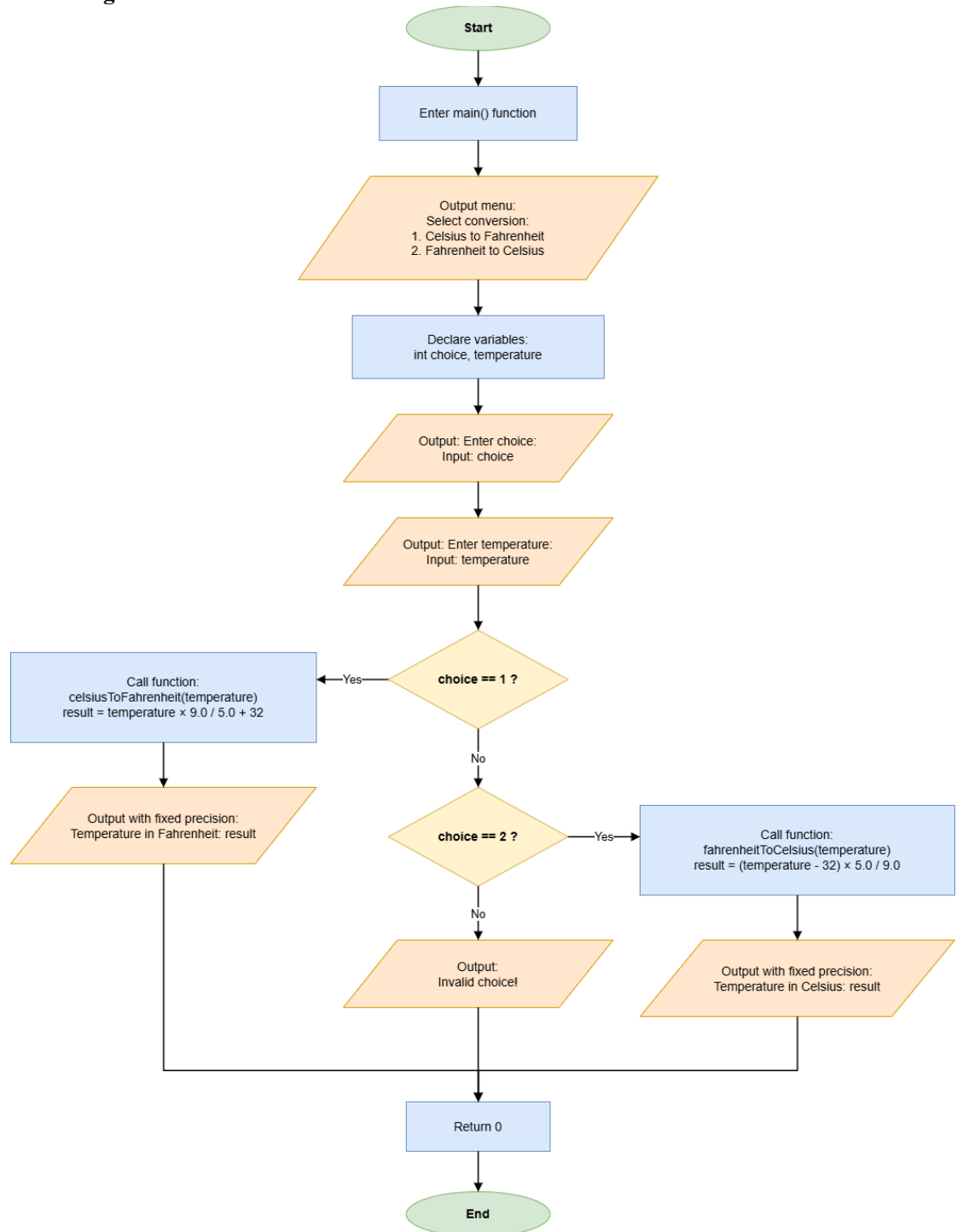
- 3) Call the two functions from `main()` depending on the user's choice.

```
if (choice == 1)
{
    cout << fixed << setprecision(2);
    cout << "Temperature in Fahrenheit: " <<
celsiusToFahrenheit(temperature) << endl;
}
else if (choice == 2)
{
    cout << fixed << setprecision(2);
    cout << "Temperature in Celsius: " <<
fahrenheitToCelsius(temperature) << endl;
}
```

- 4) Protection from invalid choice.

```
else
{
    cout << "Invalid choice!" << endl;
}
```

2. Flow Diagram:



3. Results (Output Screenshot):

The screenshot displays the Visual Studio Code interface with a C++ project named 'Experiment_3'. The file explorer on the left shows a directory structure with 'src' containing several task files (Task_1.exe to Task_8.exe) and 'bin' containing corresponding executables. The main editor shows the source code for 'Experiment_3_Task_5.cpp', which implements a temperature conversion program. The code includes a `fahrenheitToCelsius` function and a `main` function that prompts the user to select a conversion direction (1 for Celsius to Fahrenheit, 2 for Fahrenheit to Celsius) and then enters the temperature value. The output window at the bottom shows the program's execution: it prompts for a conversion choice, the user enters '1', then a temperature of '36.5', resulting in 'Temperature in Fahrenheit: 96.88'. Subsequently, the user enters '2' and a temperature of '98.6', resulting in 'Temperature in Celsius: 36.67'.

```
src > Experiment_3_Task_5.cpp > fahrenheitToCelsius(double fahrenheit);
double fahrenheitToCelsius(double fahrenheit);

8 int main()
9 {
10     cout << "Select conversion:" << endl;
11     cout << "1. Celsius to Fahrenheit" << endl;
12     cout << "2. Fahrenheit to Celsius" << endl;
13
14     int choice, temperature;
15     cout << "Enter choice: ";
16     cin >> choice;
17     cout << "Enter temperature: ";
18     cin >> temperature;
19
20     if (choice == 1)
21     {
22         cout << fixed << setprecision(2);
23         cout << "Temperature in Fahrenheit: " << celsiusToFahrenheit(temperature) << endl;
24     }
25     else if (choice == 2)
26     {
27         cout << fixed << setprecision(2);
28         cout << "Temperature in Celsius: " << fahrenheitToCelsius(temperature) << endl;
29     }
30     else
31     {
32         cout << "Invalid choice!" << endl;
33     }
34
35     return 0;
36 }
```

```
PS D:\OneDrive\Desktop\Experiment_3> .\bin\Experiment_3_Task_5.exe
Select conversion:
1. Celsius to Fahrenheit
2. Fahrenheit to Celsius
Enter choice: 1
Enter temperature: 36.5
Temperature in Fahrenheit: 96.88
PS D:\OneDrive\Desktop\Experiment_3> .\bin\Experiment_3_Task_5.exe
Select conversion:
1. Celsius to Fahrenheit
2. Fahrenheit to Celsius
Enter choice: 2
Enter temperature: 98.6
Temperature in Celsius: 36.67
PS D:\OneDrive\Desktop\Experiment_3>
```

Task 6: Prime Number Checker (Boolean Function)

1. Description:

- 1) Write a function `bool isPrime(int n)` that returns `true` if a number is prime and `false` otherwise.

```
bool isPrime(int n)
{
    for (int i = 2; i <= n / 2; i++)
    {
        if (n % i == 0)
            return false;
    }
    return true;
}
```

- 2) Input the number to be checked.

```
int number;
cout << "Enter a number: ";
cin >> number;
```

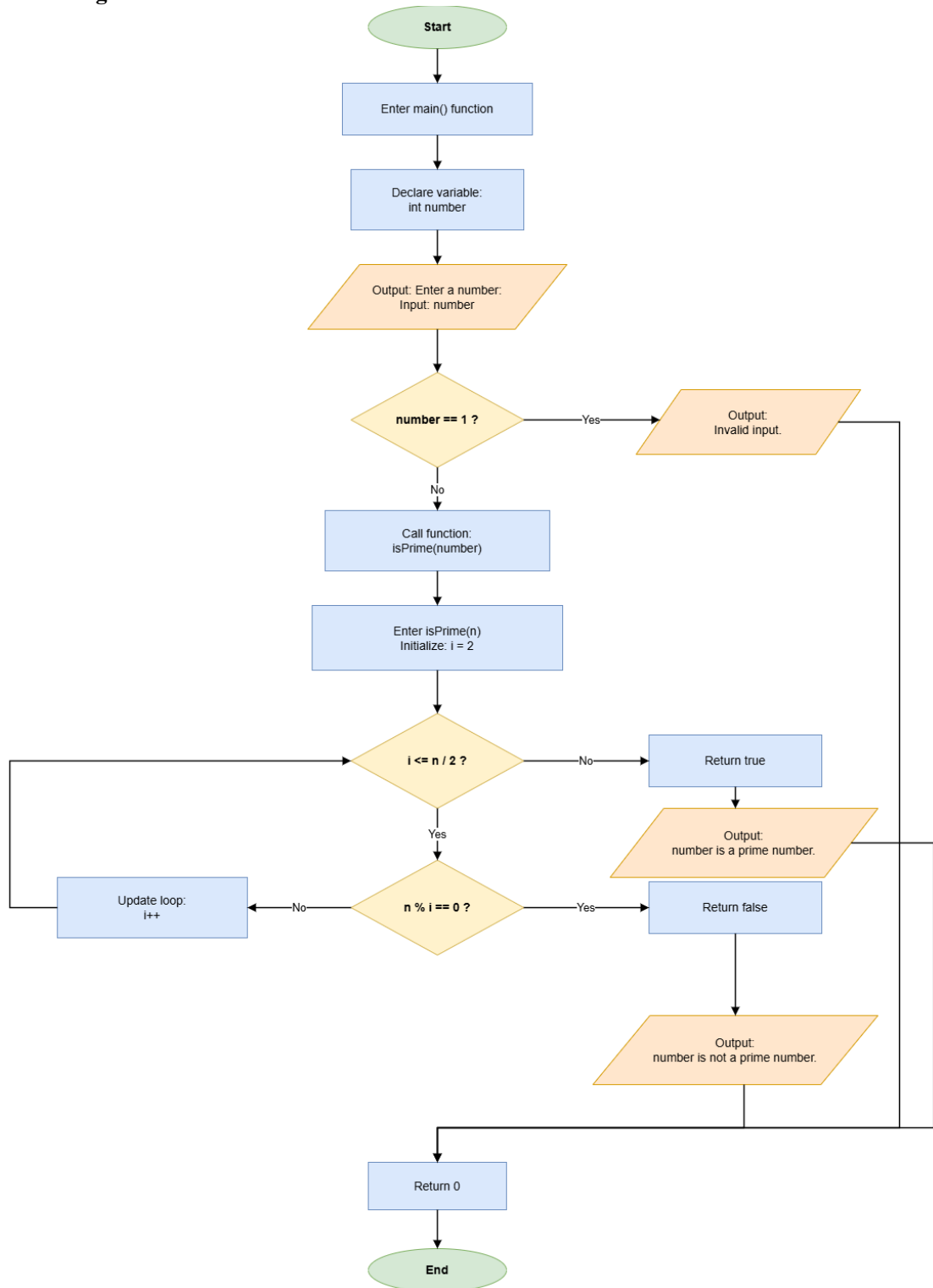
- 3) Protection when number is 1.

```
if (number == 1)
{
    cout << "Invalid input." << endl;
}
```

- 4) Call `isPrime()` in `main()` to check the user's input.

```
else if (isPrime(number))
{
    cout << number << " is a prime number." << endl;
}
else
{
    cout << number << " is not a prime number." << endl;
}
```

2. Flow Diagram:



3. Results (Output Screenshot):

The screenshot displays the Visual Studio Code interface with a C++ project named 'Experiment_3'. The file explorer on the left shows the project structure, including source files and executables. The main editor window displays the code for 'Experiment_3_Task_6.cpp', which implements a function 'isPrime' to check if a number is prime. The code uses a simple loop to test divisibility. The output console at the bottom shows the program's execution results for three test cases: 17 (prime), 18 (not prime), and 1 (invalid input). The interface is in Chinese, with the file explorer and output console labels in that language.

```
src > Experiment_3_Task_6.cpp > isPrime(int)
1  #include <iostream>
2  using namespace std;
3
4  bool isPrime(int n);
5
6  int main()
7  {
8      int number;
9      cout << "Enter a number: ";
10     cin >> number;
11
12     if (number == 1)
13     {
14         cout << "Invalid input." << endl;
15     }
16     else if (isPrime(number))
17     {
18         cout << number << " is a prime number." << endl;
19     }
20     else
21     {
22         cout << number << " is not a prime number." << endl;
23     }
24
25     return 0;
26 }
27
28 bool isPrime(int n)
29 {
30     for (int i = 2; i <= n; i++)
31     {
32         if (n % i == 0)
33             return false;
34     }
35     return true;
36 }
```

PS D:\OneDrive\Desktop\Experiment_3> .\bin\Experiment_3_Task_6.exe
Enter a number: 17
17 is a prime number.
PS D:\OneDrive\Desktop\Experiment_3> .\bin\Experiment_3_Task_6.exe
Enter a number: 18
18 is not a prime number.
PS D:\OneDrive\Desktop\Experiment_3> .\bin\Experiment_3_Task_6.exe
Enter a number: 1
Invalid input.
PS D:\OneDrive\Desktop\Experiment_3>

Task 7: Mathematical Library Exploration (cmath)

1. Description:

- 1) Input a positive number.

```
double n;  
cout << "Enter a number: ";  
cin >> n;
```

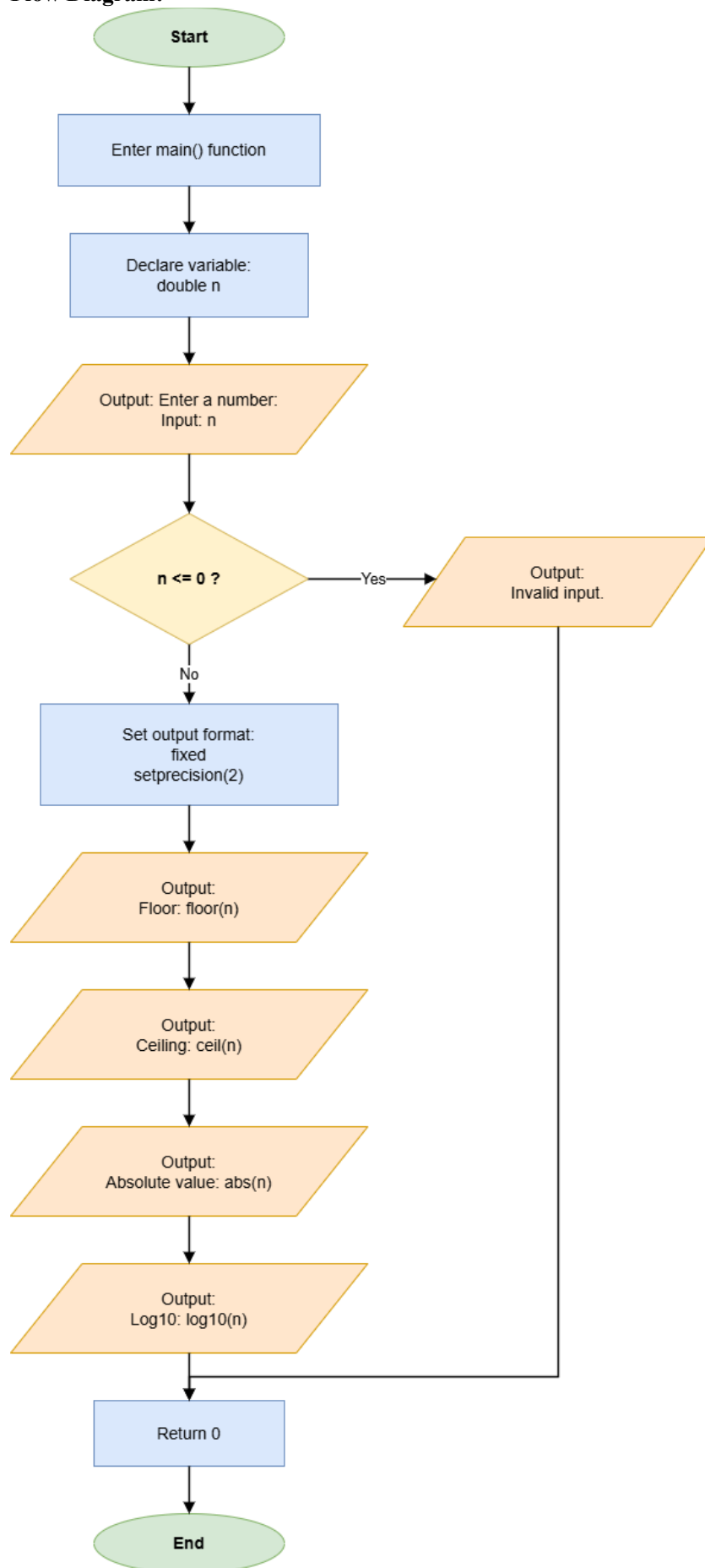
- 2) When the input is not positive, print "Invalid output."

```
if (n <= 0)  
{  
    cout << "Invalid input." << endl;  
}
```

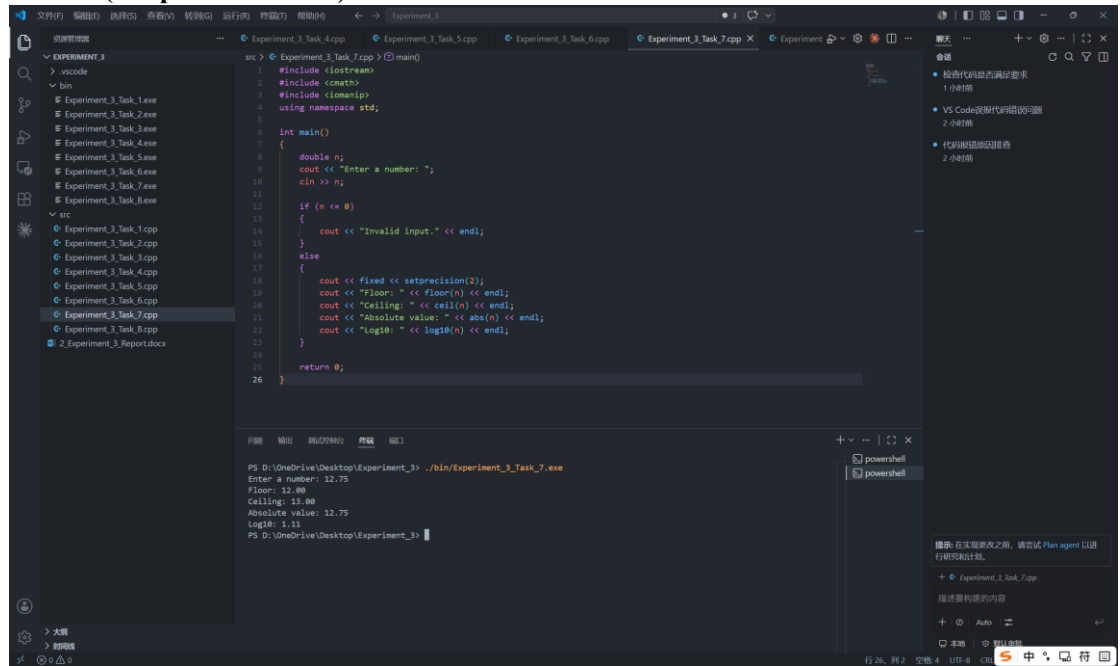
- 3) Use floor(), ceil(), abs(), and log10() to process the input number.

```
else  
{  
    cout << fixed << setprecision(2);  
    cout << "Floor: " << floor(n) << endl;  
    cout << "Ceiling: " << ceil(n) << endl;  
    cout << "Absolute value: " << abs(n) << endl;  
    cout << "Log10: " << log10(n) << endl;  
}
```

2. Flow Diagram:



3. Results (Output Screenshot):



The screenshot displays the Visual Studio Code interface with a C++ program open in the editor. The file explorer on the left shows a project named 'EXPERIMENT_3' with various task files. The main editor window shows the source code for 'Experiment_3_Task_7.cpp'. The code includes headers for `<iostream>` and `<cmath>`, uses the `std` namespace, and defines a `main` function. The program prompts the user to 'Enter a number:', reads the input, and then outputs several mathematical results: the floor, ceiling, absolute value, and base-10 logarithm of the input.

```
1 #include <iostream>
2 #include <cmath>
3 #include <iomanip>
4 using namespace std;
5
6 int main()
7 {
8     double n;
9     cout << "Enter a number: ";
10    cin >> n;
11
12    if (n <= 0)
13    {
14        cout << "Invalid input." << endl;
15    }
16    else
17    {
18        cout << fixed << setprecision(2);
19        cout << "Floor: " << floor(n) << endl;
20        cout << "Ceiling: " << ceil(n) << endl;
21        cout << "Absolute value: " << abs(n) << endl;
22        cout << "Log10: " << log10(n) << endl;
23    }
24
25    return 0;
26 }
```

The output window at the bottom shows the execution results for the program. The prompt 'Enter a number:' is followed by the input '12.75'. The program then outputs the following values: Floor: 12.00, Ceiling: 13.00, Absolute value: 12.75, and Log10: 1.11.

```
PS D:\OneDrive\Desktop\Experiment_3> .\bin\Experiment_3_Task_7.exe
Enter a number: 12.75
Floor: 12.00
Ceiling: 13.00
Absolute value: 12.75
Log10: 1.11
PS D:\OneDrive\Desktop\Experiment_3>
```

Task 8: Simple Menu System (Modular Program)

1. Description:

- 1) Create the three functions.

```
long calculatePower(int base, int exp)
{
    long result = 1;
    for (int i = 0; i < exp; i++)
    {
        result *= base;
    }
    return result;
}

double getCircleArea(double radius)
{
    const double PI = 3.14159;
    return PI * radius * radius;
}

bool isPrime(int n)
{
    for (int i = 2; i <= n / 2; i++)
    {
        if (n % i == 0)
            return false;
    }
    return true;
}
```

- 2) Print the menu.

```
cout << "Menu:" << endl;
cout << "1. Power Calculation" << endl;
cout << "2. Circle Area" << endl;
cout << "3. Prime Check" << endl;
```

- 3) Input the choice.

```
int choice;
cout << "Enter choice: ";
cin >> choice;
```

- 4) Use switch-case to call different functions depending on the user's choice.

```
switch (choice)
{
    case 1:
    {
        int base;
        cout << "Enter number: ";
        cin >> base;

        cout << "Square: " << calculatePower(base, 2) << endl;
        cout << "Cube: " << calculatePower(base, 3) << endl;
        break;
    }
}
```



```

}

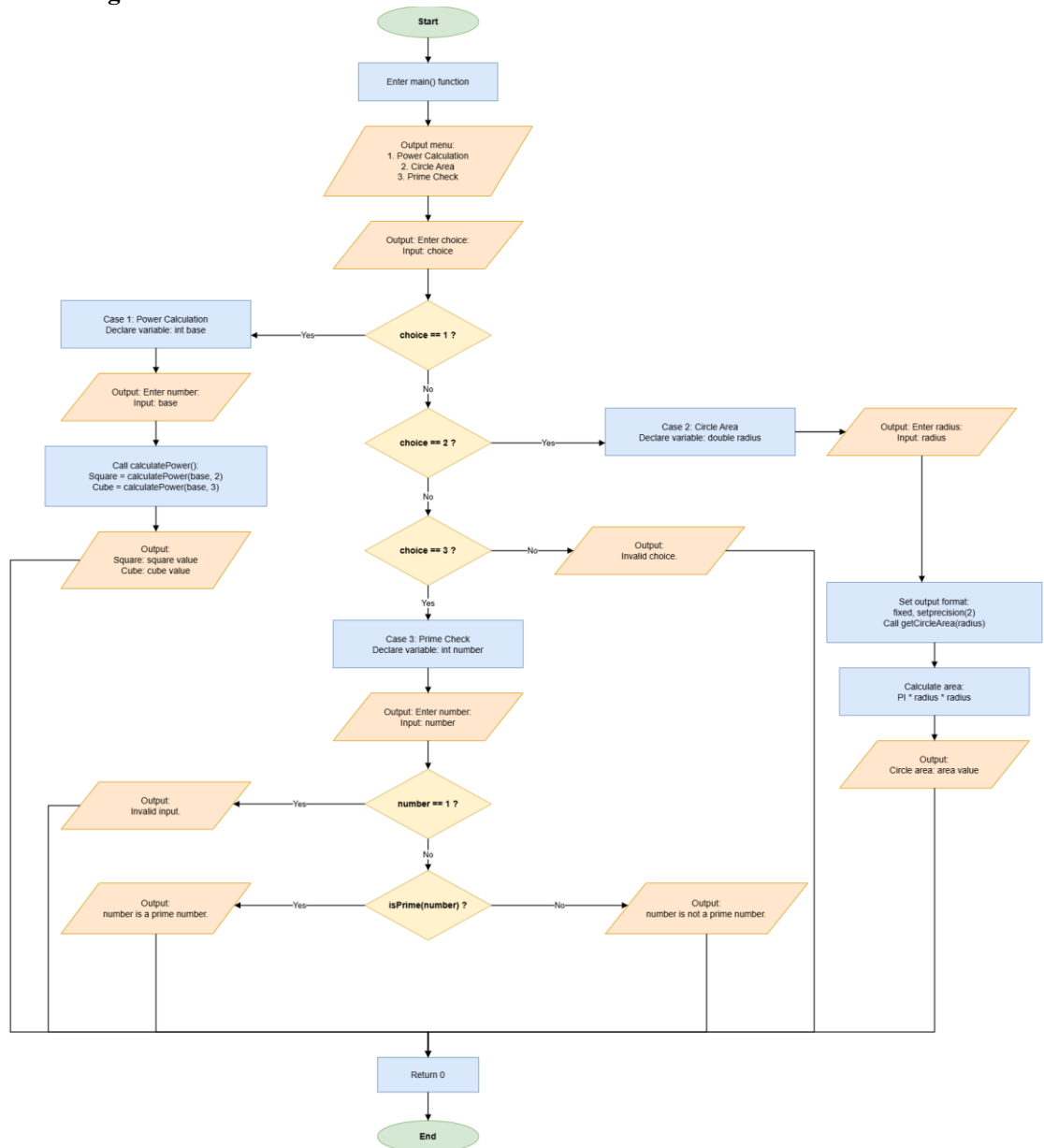
case 2:
{
    double radius;
    cout << "Enter radius: ";
    cin >> radius;
    cout << fixed << setprecision(2);
    cout << "Circle area: " << getCircleArea(radius) << endl;
    break;
}

case 3:
{
    int number;
    cout << "Enter number: ";
    cin >> number;

    if (number == 1)
    {
        cout << "Invalid input." << endl;
    }
    else if (isPrime(number))
    {
        cout << number << " is a prime number." << endl;
    }
    else
    {
        cout << number << " is not a prime number." << endl;
    }
    break;
}
default:
    cout << "Invalid choice." << endl;
}

```

2. Flow Diagram:



3. Results (Output Screenshot):

The screenshot displays the Visual Studio Code interface with a C++ project named 'Experiment_3'. The file explorer on the left shows a directory structure with files like 'Task_1.exe' through 'Task_8.exe' and 'Task_1.cpp' through 'Task_8.cpp'. The main editor window shows the code for 'Experiment_3_Task_8.cpp', which implements a menu-driven program with options for Power Calculation, Circle Area, Prime Check, and an invalid choice. The output window at the bottom shows the execution of the program, demonstrating the menu flow and calculations for different inputs.

```
int main()
{
    switch (choice)
    {
        case 1:
            // Power Calculation
            break;
        case 2:
            // Circle Area
            break;
        case 3:
            // Prime Check
            break;
        case 4:
            // Invalid choice
            break;
    }
}
```

Output:

```
PS D:\OneDrive\Desktop\Experiment_3> ./bin/Experiment_3_Task_8.exe
Menu:
1. Power Calculation
2. Circle Area
3. Prime Check
Enter choice: 1
Enter number: 4
Square: 16
Cube: 64
PS D:\OneDrive\Desktop\Experiment_3> ./bin/Experiment_3_Task_8.exe
Menu:
1. Power Calculation
2. Circle Area
3. Prime Check
Enter choice: 2
Enter radius: 5
Circle area: 78.54
PS D:\OneDrive\Desktop\Experiment_3> ./bin/Experiment_3_Task_8.exe
Menu:
1. Power Calculation
2. Circle Area
3. Prime Check
Enter choice: 3
Enter number: 11
11 is a prime number.
PS D:\OneDrive\Desktop\Experiment_3> ./bin/Experiment_3_Task_8.exe
Menu:
1. Power Calculation
2. Circle Area
3. Prime Check
Enter choice: 4
Invalid choice.
PS D:\OneDrive\Desktop\Experiment_3>
```

IV. Extended Questions

1. In Task 1, the function `displayHeader()` is declared before `main()` and defined after `main()`. What may happen if `displayHeader()` is called in `main()` before it is declared?

If `displayHeader()` is called before the compiler has seen either its function prototype or its full definition, the program will not compile. The compiler cannot recognize the function name at that point, so it may report an error such as “`displayHeader` was not declared in this scope.” To avoid this problem, write a prototype such as `void displayHeader();` before `main()`, or place the complete function definition before `main()`.

2. Read the following code and answer the questions.

```
#include <iostream>
using namespace std;

int number = 10;    // global variable

void showNumber(int number) {
    cout << "In function: " << number << endl;
}

int main() {
    int number = 5;    // local variable

    showNumber(number);
    cout << "In main (local): " << number << endl;
    cout << "In global: " << ::number << endl;

    return 0;
}
```

Questions:

- 1) What is the output of this program?

In function: 5

In main (local): 5

In global: 10

- 2) Why does the program produce this output? Explain the roles of the local variable, the global variable, the function parameter, and `::number`.

The global variable `number` is initialized to 10, but inside `main()` another local variable with the same name is declared and initialized to 5. The local variable has higher priority within `main()`, so `showNumber(number)` passes the local value 5 to the function. In `showNumber(int number)`, the function parameter is also named `number`, and it receives the argument value 5, so the first output line is 5. The statement `cout << number` inside `main()` still refers to the local variable, so the second line is also 5. The expression `::number` uses the scope resolution operator to explicitly access the global variable, so the last line prints 10.

V. Summary and Reflections

In this experiment, I practiced the basic use of functions in C++ through eight programming tasks. The tasks covered void functions, data-returning functions, functions with parameters, Boolean functions, multiple function calls, and the use of standard library functions from `<cmath>`. By writing programs for header output, power calculation, maximum value comparison, circle area calculation, temperature conversion, prime number checking, mathematical function processing, and a menu-driven program, I learned how to divide a program into smaller modules and call the correct function according to the program logic.

The main techniques used in this experiment included function declaration, function definition, parameter passing, return values, conditional statements, loops, switch-case selection, constants, and formatted output with `fixed` and `setprecision(2)`. The final results show that each function can complete its own task and return or display the expected result. This helped me understand that modular programming can make code clearer, easier to debug, and easier to reuse. During the experiment, I also paid attention to invalid input handling, such as checking menu choices and prime-number input.